



Basi di Dati
Progetto A.A. 2023/2024

Chat multicanale

0306834
Valentina Jin

Indice

1. Descrizione del Minimondo.....	2
2. Analisi dei Requisiti.....	3
3. Progettazione concettuale.....	6
4. Progettazione logica.....	11
5. Progettazione fisica.....	18

1. Descrizione del Minimondo

- | | |
|----|--|
| 1 | Si vuole realizzare un sistema informativo per consentire ai lavoratori di una azienda di |
| 2 | scambiare messaggi legati ai progetti a cui stanno attualmente lavorando. |
| 3 | Il sistema prevede tre livelli di utenza: gli amministratori, i capi progetto, i dipendenti. |
| 4 | Gli amministratori hanno la possibilità di gestire quali utenti sono capi progetto. |
| 5 | I capi progetto possono creare un numero arbitrario di canali di comunicazione ed invitare |
| 6 | al loro interno tutti i dipendenti che cooperano sulle attività del progetto. |
| 7 | All'interno di un canale gli utenti possono inviare messaggi che possono essere letti da tutti |
| 8 | gli altri appartenenti al canale. I messaggi sono organizzati in pagine e gli utenti possono |
| 9 | visualizzare, una per una, le pagine della conversazione. |
| 10 | Un utente del sistema ha la possibilità di rispondere pubblicamente in un canale. In questa |
| 11 | risposta, può decidere di riferire un messaggio precedentemente inviato, così che il suo |
| 12 | messaggio appaia come risposta ad una parte specifica della comunicazione. Allo stesso |
| 13 | modo, partendo da un qualsiasi messaggio, l'utilizzatore può decidere di rispondere in |
| 14 | modo privato. Questa risposta privata aprirà un canale di discussione privato tra lui e il |
| 15 | mittente del messaggio cui si sta rispondendo. I project manager possono sempre accedere |
| 16 | (in sola lettura) a tutte le discussioni private nate nei canali dei progetti di cui sono |
| 17 | responsabili. |
| 18 | Un utente può accedere in qualsiasi momento a tutti i canali di cui fa parte, e a tutte le |
| 19 | conversazioni private (organizzate per canale) che ha creato o in cui è stato coinvolto. |

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili.

Linea	Termine	Nuovo termine	Motivo correzione
7,8	Utenti	Capi progetto e Dipendenti	La scrittura dei messaggi è abilitata solo ai dipendenti e capi progetto e non l'utenza in generale che comprende anche gli amministratori.
10	Utente del sistema	Capo progetto e Dipendente	La scrittura dei messaggi è abilitata solo ai dipendenti e capi progetto e non l'utenza in generale che comprende anche gli amministratori.
13	Utilizzatore	Capo progetto o Dipendente	Specifico chi può essere l'utilizzatore di un messaggio particolare all'interno di un canale.
14	Lui	Rispondente	Non c'è differenza semantica nel minimondo di riferimento.
15	Project manager	Capi progetto	Il ruolo che i project manager svolgono è lo stesso dei capi progetto, per coerenza usiamo lo stesso termine di prima.

Specifica disambiguata

Si vuole realizzare un sistema informativo per consentire ai lavoratori di una azienda di scambiare messaggi legati ai progetti a cui stanno attualmente lavorando.

Il sistema prevede tre livelli di utenza: gli amministratori, i capi progetto, i dipendenti.

Gli amministratori hanno la possibilità di gestire quali utenti sono capi progetto.

I capi progetto possono creare un numero arbitrario di canali di comunicazione ed invitare al loro interno tutti i dipendenti che cooperano sulle attività del progetto.

All'interno di un canale i capi progetto e dipendenti possono inviare messaggi che possono essere letti da tutti gli altri appartenenti al canale. I messaggi sono organizzati in pagine e i capi progetto e dipendenti possono visualizzare, una per una, le pagine della conversazione.

Capi progetto e dipendenti hanno la possibilità di rispondere pubblicamente in un canale. In questa risposta, può decidere di riferire un messaggio precedentemente inviato, così che il suo messaggio appaia come risposta ad una parte specifica della comunicazione. Allo stesso modo, partendo da un qualsiasi messaggio, il capo progetto o il dipendente può decidere di rispondere in modo privato. Questa risposta privata aprirà un canale di discussione privato tra il rispondente e il mittente del messaggio cui si sta rispondendo. I capi progetto possono sempre accedere (in sola lettura) a tutte le discussioni private nate nei canali dei progetti di cui sono responsabili.

Un utente può accedere in qualsiasi momento a tutti i canali di cui fa parte, e a tutte le conversazioni private (organizzate per canale) che ha creato o in cui è stato coinvolto.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Lavoratore	Impiegato dell'azienda	Utente	Messaggio, Progetto
Amministratore	Impiegato incaricato di coordinare i capi progetto		Lavoratore, Capi progetto
Capo progetto	Impiegato incaricato di coordinare tutte le attività afferenti a determinati progetti	Project manager	Lavoratore, Canale di comunicazione, Canale di discussione privata, Messaggio, Progetto
Dipendente	Impiegato incaricato di svolgere determinate attività dei progetti a cui è stato assegnato		Lavoratore, Canale di comunicazione, Canale di discussione privata, Messaggio, Progetto
Canale di comunicazione	Mezzo che permette lo scambio di messaggi tra i lavoratori che lavorano su un determinato progetto	Canale di discussione, Conversazione	Messaggio, Progetto, Lavoratore
Progetto	Insieme strutturato di attività e compiti assegnati ad un gruppo di lavoratori		Capo progetto, Dipendente, Canale di comunicazione
Messaggio	Testo di lunghezza arbitraria che può essere scambiato tra i dipendenti che lavorano su un determinato progetto	Risposta	Capo progetto, Dipendente, Canale di comunicazione, Canale di discussione privata
Canale di discussione privata	Canale di comunicazione aperto tra due impiegati responsabili di un determinato progetto, visibile solo a loro e al capo progetto.		Canale di comunicazione, Dipendenti, Capo progetto, Messaggio

Raggruppamento dei requisiti in insiemi omogenei

Frasi relative a Lavoratore

Il sistema prevede tre livelli di utenza: gli amministratori, i capi progetto, i dipendenti.

Frasi relative a Amministratore

Gli amministratori hanno la possibilità di gestire quali utenti sono capi progetto.

Frase relative a Capo progetto

I capi progetto possono creare un numero arbitrario di canali di comunicazione ed invitare al loro interno tutti i dipendenti che cooperano sulle attività del progetto.

All'interno di un canale i capi progetto e dipendenti possono inviare messaggi che possono essere letti da tutti gli altri appartenenti al canale. I messaggi sono organizzati in pagine e i capi progetto e dipendenti possono visualizzare, una per una, le pagine della conversazione.

Capi progetto e dipendenti hanno la possibilità di rispondere pubblicamente in un canale.

Allo stesso modo, partendo da un qualsiasi messaggio, il capo progetto o il dipendente può decidere di rispondere in modo privato.

I capi progetti possono sempre accedere (in sola lettura) a tutte le discussioni private nate nei canali dei progetti di cui sono responsabili.

Frase relative a Dipendente

All'interno di un canale i capi progetto e dipendenti possono inviare messaggi che possono essere letti da tutti gli altri appartenenti al canale. I messaggi sono organizzati in pagine e i capi progetto e dipendenti possono visualizzare, una per una, le pagine della conversazione.

Capi progetto e dipendenti hanno la possibilità di rispondere pubblicamente in un canale.

Allo stesso modo, partendo da un qualsiasi messaggio, il capo progetto o il dipendente può decidere di rispondere in modo privato.

Frase relative a Messaggio

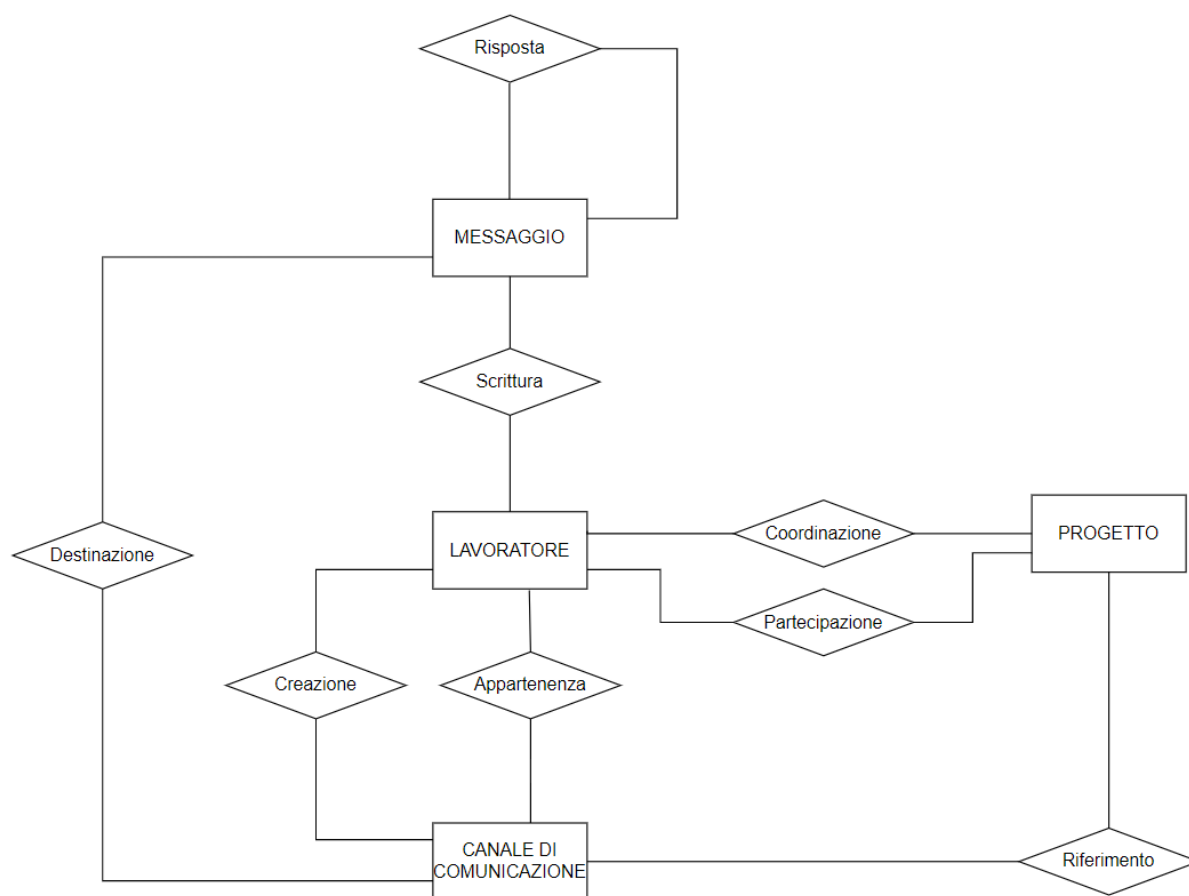
I messaggi sono organizzati in pagine e i capi progetto e dipendenti possono visualizzare, una per una, le pagine della conversazione.

In questa risposta, può decidere di riferire un messaggio precedentemente inviato, così che il suo messaggio appaia come risposta ad una parte specifica della comunicazione. Questa risposta privata aprirà un canale di discussione privato tra il rispondente e il mittente del messaggio cui si sta rispondendo.

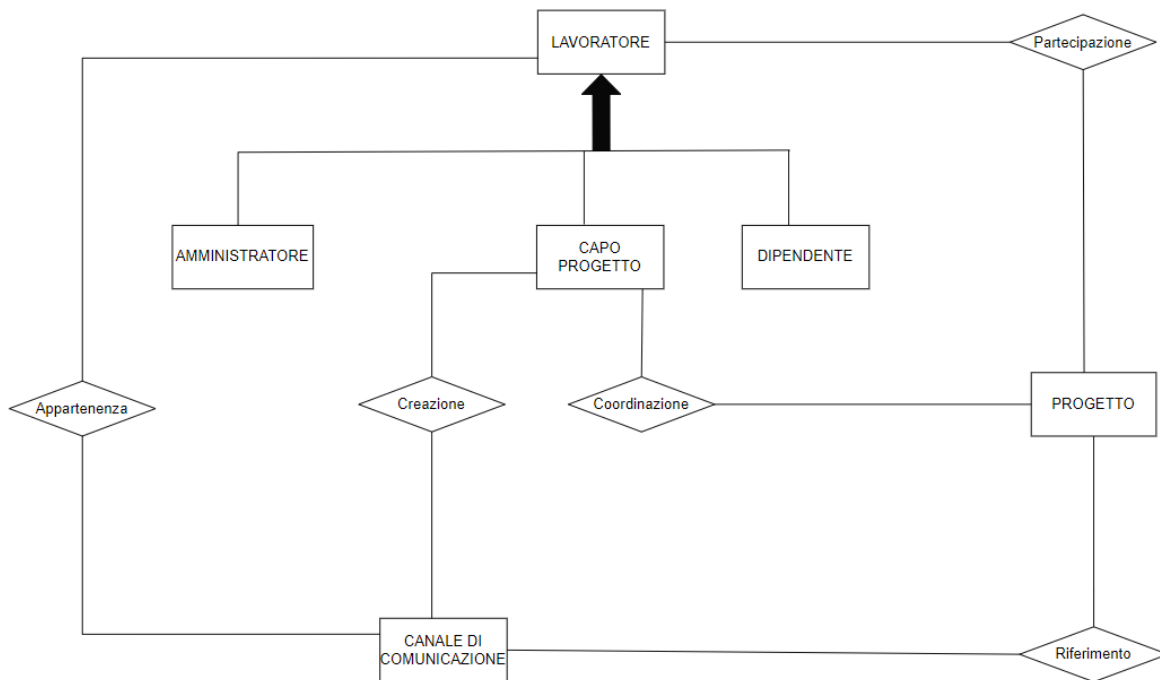
3. Progettazione concettuale

Costruzione dello schema E-R

Organizziamo la progettazione concettuale della nostra base di dati adottando la strategia top-down, ovvero partiamo con la costruzione di uno schema iniziale che descrive tutte le specifiche con pochi concetti molto astratti, poi lo schema verrà man mano raffinato aumentando i dettagli. Innanzitutto, costruiamo lo schema di base, che inquadra tutti i concetti chiave della specifica.

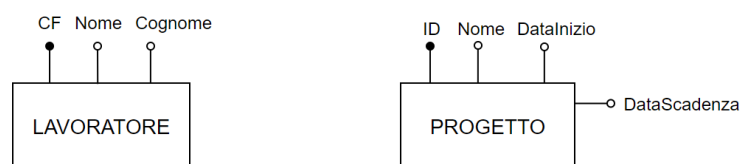


Adesso ci dedichiamo al raffinamento dell'entità LAVORATORE, si evince dalla specifica la presenza di tre tipi di utenti: Amministratore, Capo progetto e Dipendente, in questo caso specifico possiamo trasformare l'entità LAVORATORE in una gerarchia di generalizzazione per distinguere i vari tipi di utenti. In seguito, viene mostrato lo schema parziale dopo la specializzazione.

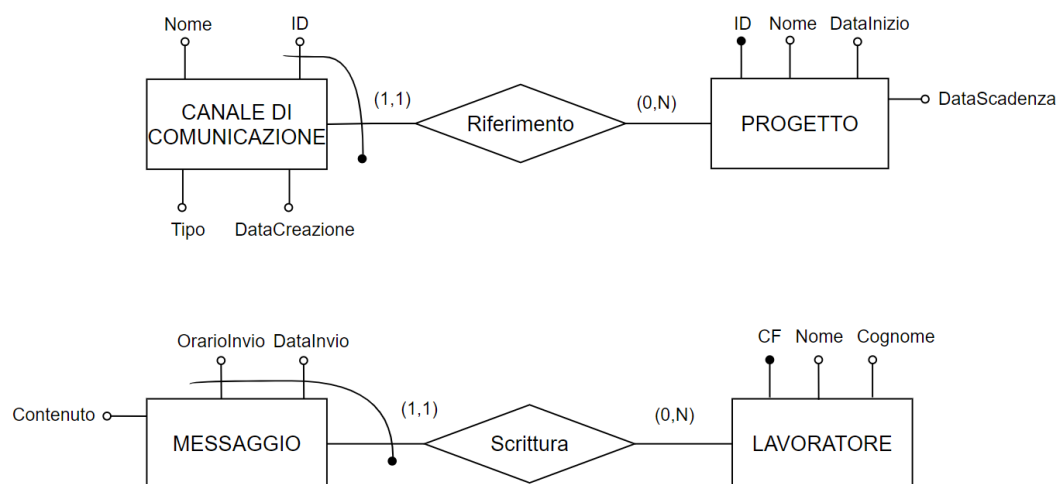


In un secondo momento ci dedichiamo alla definizione degli eventuali attributi per ogni entità e relazione, e specifichiamo le cardinalità assegnate alle varie associazioni. In particolare, nella descrizione del minimondo non viene specificato esplicitamente gli identificatori e gli attributi assegnati ai concetti chiavi, per cui scegliamo alcuni con il criterio di far funzionare correttamente l'applicazione.

Innanzitutto illustriamo le entità forti con i relativi attributi:

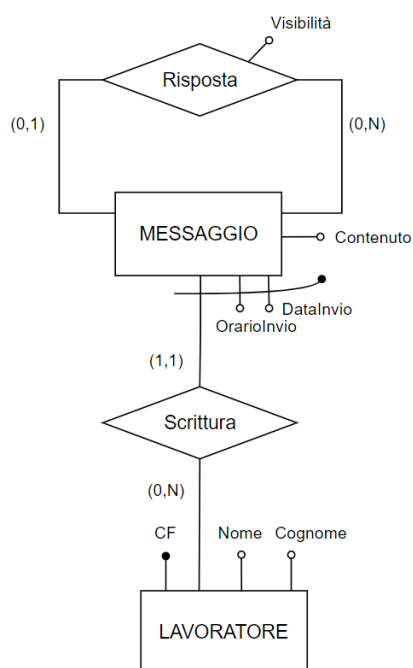


Poi illustriamo le entità deboli che hanno bisogno dell'identificazione esterna:



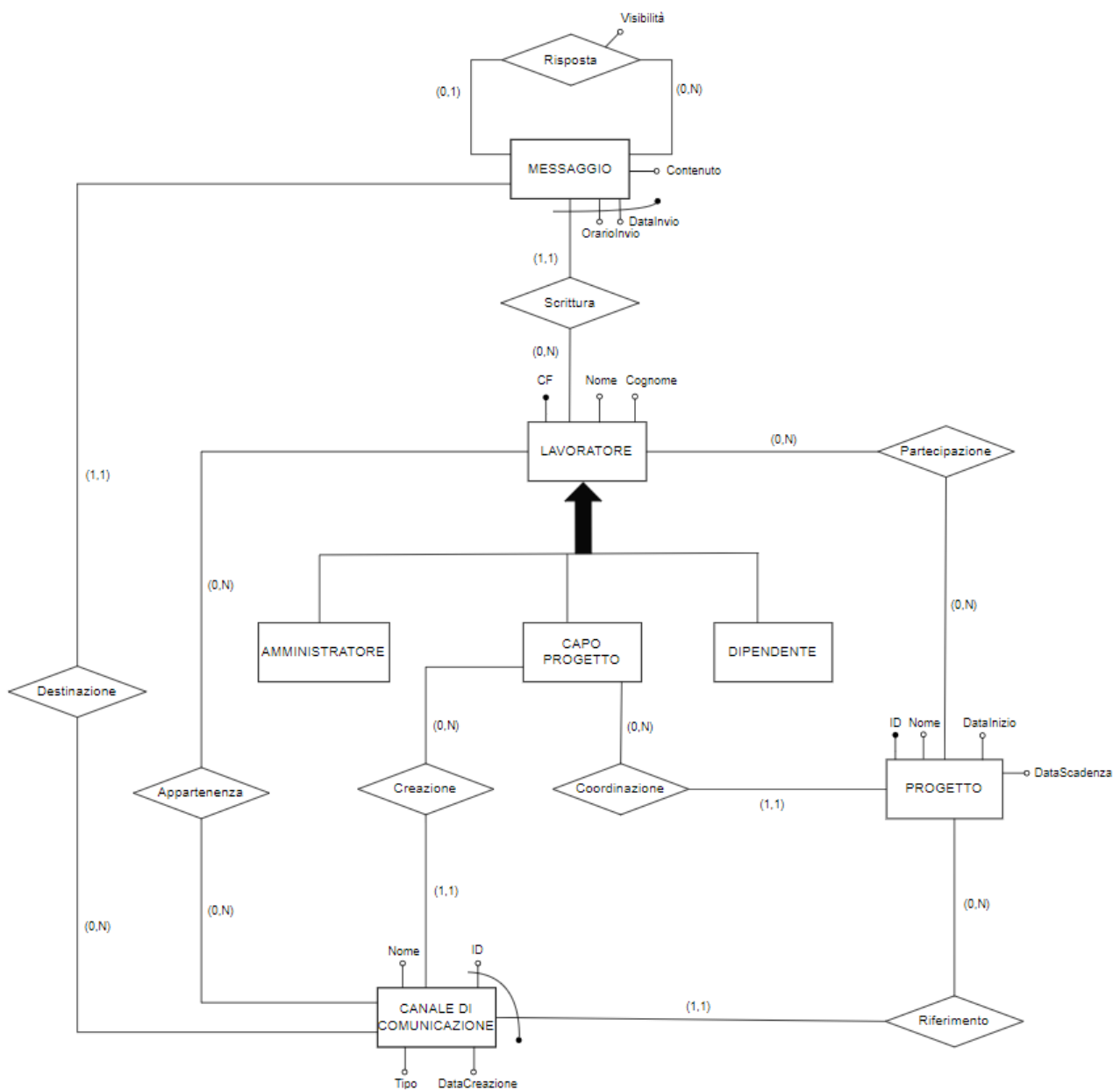
Dove specifichiamo anche le cardinalità, in particolare un canale di comunicazione è riferito esclusivamente a un progetto, invece un progetto può avere un numero arbitrario di canale di comunicazione, per cui CANALE DI COMUNICAZIONE e PROGETTO sono collegate tramite la relazione “Riferimento” di tipo “uno a molti”. Nello stesso modo, un messaggio specifico verrà scritto da un lavoratore e inviato in un dato orario e data, invece un lavoratore può scrivere e inviare più messaggi.

In seguito, andiamo a specificare l’associazione ricorsiva presente nell’entità MESSAGGIO, dove un messaggio può essere la risposta di un altro messaggio, da cui proviene la cardinalità (0,1), poi un messaggio può ricevere un numero arbitrario di risposte, per cui viene assegnata la cardinalità (0,N) dal altro lato. Inoltre viene aggiunto l’attributo “Visibilità” all’associazione “Risposta”, che serve per distinguere se un messaggio risposta avviene in modo pubblico o in modo privato.



Una volta specificato i relativi attributi associati alle entità, possiamo arricchire il nostro schema di base per ottenere lo schema finale.

Integrazione finale



Regole aziendali

1. Un amministratore non può accedere e inviare messaggi nei canali di comunicazione.
2. Oltre i capi progetto, solo i messaggi inviati come risposta privata degli altri possono creare un canale di comunicazione tra il rispondente e il mittente del messaggio originario, tale canale avrà lo stesso progetto di riferimento del canale da cui ha avuto origine.
3. La data di inizio di un progetto deve essere precedente rispetto alla data di scadenza di tale progetto.

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Messaggio	Testo inviato in un canale di comunicazione in modo privato o pubblico, relativo ad un progetto	DataInvio, OrarioInvio, Contenuto	CF, DataInvio, OrarioInvio
Lavoratore	Impiegato dell'azienda	CF, Nome, Cognome	CF
Amministratore	Impiegato incaricato di coordinare i capi progetto	CF, Nome, Cognome	CF
Capo progetto	Impiegato incaricato di coordinare tutte le attività afferenti a determinati progetti	CF, Nome, Cognome	CF
Dipendente	Impiegato incaricato di svolgere determinate attività dei progetti a cui è stato assegnato	CF, Nome, Cognome	CF
Progetto	Insieme strutturato di attività e compiti assegnati ad un gruppo di lavoratori	ID, Nome, DataInizio, DataScadenza	ID
Canale di comunicazione	Mezzo che permette lo scambio di messaggi tra i lavoratori che lavorano su un determinato progetto	IDProgetto, IDCanale, Nome	IDProgetto, IDCanale

4. Progettazione logica

Volume dei dati

1. Si assume un periodo di validità dei dati di 3 anni dopo la scadenza del progetto, dopo i quali tutti i dati vengono definitivamente eliminati dal database.
2. Si assume che per ogni progetto viene creato 10 canali di comunicazione.
3. Si assume che in un canale di comunicazione viene mandato circa 100 messaggi al giorno.

Concetto nello schema	Tipo ¹	Volume atteso
Lavoratore	E	500
Amministratore	E	20
Capo progetto	E	60
Dipendente	E	420
Progetto	E	80
Canale di comunicazione	E	800
Messaggio	E	800000
Risposta	R	270000
Scrittura	R	530000
Destinazione	R	800000
Partecipazione	R	600
Appartenenza	R	1200
Creazione	R	800
Coordinazione	R	80
Riferimento	R	800

Tavola delle operazioni

Nella tavola delle operazioni sono state riportate le operazioni più significative dal punto di vista applicativo. Nella specifica viene sottolineato che il sistema informativo deve consentire i lavoratori di scambiare messaggi legati ai progetti, per cui non sono riportate nella tabella le operazioni come inserimento di un nuovo lavoratore, o cancellazione di un lavoratore, in quanto ritenute operazioni banali.

Cod.	Descrizione	Frequenza attesa
AM1	Assegnare un capo progetto ad un progetto	20/mese
C1	Crea un canale di comunicazione	20/mese
C2	Visualizza i partecipanti a un progetto	50/mese
C3	Aggiunge i lavoratori al canale	100/mese
D1	Scriva un nuovo messaggio	1000/giorno
D2	Risponde un specifico messaggio	335/giorno

¹ Indicare con E le entità, con R le relazioni

D3	Lettura di messaggi in un canale	1000/giorno
D4	Visualizza i canali a cui fa parte	500/mese

Costo delle operazioni

Si suppone, in questa fase di progettazione, che il costo di scrittura di un dato sia il doppio del costo in lettura.

AM1 - Assegnare un capo progetto ad un progetto

L'operazione richiede l'accesso in lettura all'entità LAVORATORE per scegliere il capo progetto per un progetto. Successivamente richiede l'accesso in scrittura all'entità CAPO PROGETTO per inserirlo nella tabella.

Costo operazione: $(1 \cdot 500 + 2 \cdot 1) \cdot 20 = 10040$ accessi/mese

C1 - Crea un canale di comunicazione

L'operazione richiede l'accesso in lettura all'entità PROGETTO, poi richiede l'accesso in scrittura alla relazione RIFERIMENTO, infine richiede l'accesso in scrittura alla CANALE DI COMUNICAZIONE per aggiungere un nuovo canale.

Costo operazione: $(1 \cdot 80 + 2 \cdot 1 + 2 \cdot 1) \cdot 20 = 1680$ accessi/mese

C2 - Visualizza i partecipanti a un progetto

L'operazione richiede l'accesso in lettura alle entità PROGETTO e LAVORATORE, e poi l'accesso in lettura alla relazione PARTECIPAZIONE.

Costo operazione: $(1 \cdot 80 + 1 \cdot 500 + 1 \cdot 600) \cdot 50 = 59000$ accessi/mese

C3 - Aggiunge i lavoratori al canale

L'operazione richiede l'accesso in lettura all'entità LAVORATORE per scegliere i lavoratori da aggiungere. Successivamente richiede l'accesso in scrittura alla relazione APPARTENENZA.

Costo operazione: $(1 \cdot 500 + 2 \cdot 1) \cdot 100 = 50200$ accessi/mese

D1 - Scrive un nuovo messaggio

L'operazione richiede l'accesso in scrittura all'entità MESSAGGIO. Successivamente richiede l'accesso in scrittura alla relazione SCRITTURA. Infine richiede l'accesso in scrittura alla relazione DESTINAZIONE.

Costo operazione: $(2 \cdot 1 + 2 \cdot 1 + 2 \cdot 1) \cdot 1000 = 6000$ accessi/giorno

D2 - Risponde un specifico messaggio

L'operazione richiede l'accesso in scrittura all'entità MESSAGGIO. Successivamente richiede l'accesso in scrittura alle rispettive relazioni: SCRITTURA, RISPOSTA e DESTINAZIONE.

Costo operazione: $(2*1 + 2*1 + 2*1 + 2*1)*335 = 2680$ accessi/giorno

D3 - Lettura di messaggi in un canale

L'operazione richiede l'accesso in lettura all'entità CANALE DI COMUNICAZIONE per scegliere il canale, poi richiede l'accesso in lettura alla relazione DESTINAZIONE e l'accesso in lettura all'entità MESSAGGIO. Dove si assume che mediamente in un canale di comunicazione è presente 1000 messaggi.

Costo operazione: $(1*800 + 1*1000 + 1*1000)*1000 = 2800000$ accessi/giorno

D4 - Visualizza i canali a cui fa parte

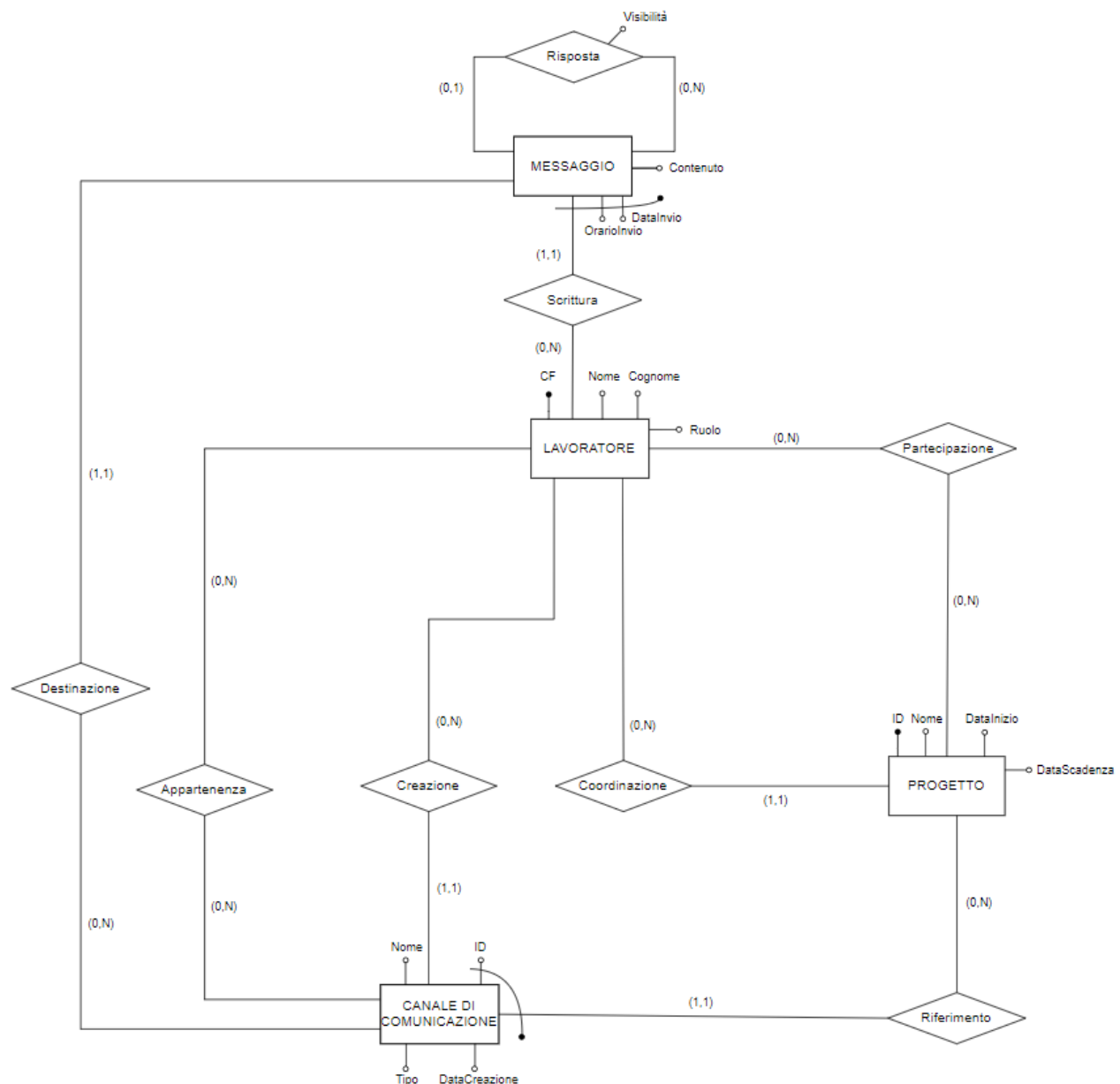
L'operazione richiede l'accesso in lettura all'entità PROGETTO, poi richiede l'accesso in lettura all'entità CANALE DI COMUNICAZIONE, e l'accesso in lettura alla relazione APPARTENENZA.

Costo operazione: $(1*80 + 1*800 + 1*1200)*500 = 1040000$ accessi/giorno

Ristrutturazione dello schema E-R

Lo schema E-R proposto è presente una generalizzazione, ovvero tra l'entità padre LAVORATORE e le entità figlie: AMMINISTRATORE, CAPO PROGETTO e DIPENDENTE. In questa fase di ristrutturazione, per eliminare tale generalizzazione, adottiamo la strategia di accorpamento delle entità figlie nell'entità padre, in particolare possiamo vedere che l'entità CAPO PROGETTO è associata alla relazione CREAZIONE e COORDINAZIONE, essendo è importante distinguere i tipi di lavoratori per rispettare le regole aziendali, per cui aggiungiamo all'entità padre un attributo "Ruolo".

Otteniamo dunque lo schema ristrutturato in tale modo:



Trasformazione di attributi e identificatori

- Per rendere più chiaro la traduzione dell'entità CANALE DI COMUNICAZIONE, l'identificatore interno "ID" viene rinominato in "IDCanale", e l'identificatore esterno "ID" viene rinominato in "IDProgetto". Inoltre tale entità accorpa in se anche l'associazione CREAZIONE, per cui viene aggiunto e rinominato un attributo "Creatore".
- Nell'associazione RISPOSTA, gli attributi vengono rinominati con l'aggiunta del termine "mittente" per rendere più chiaro a chi è riferito tale risposta, per cui diventano "Mittente", "OrarioInvioMittente" e "DataInvioMittente", invece gli identificatori riferiti al messaggio di risposta del rispondente sono rinominati con l'aggiunta del termine "Rispondente".

- L'entità PROGETTO accorpa l'associazione COORDINAZIONE, per cui viene aggiunto e rinominato un attributo "CapoProgetto" per tenere conto chi è il capo progetto che coordina un progetto specifico.

Traduzione di entità e associazioni

Partendo dallo schema ristrutturato costruiamo le seguenti relazioni per il modello relazionale.

Progetto(ID, Nome, DataInizio, DataScadenza, CapoProgetto)

CanaleDiComunicazione(IDCanale, IDProgetto, Nome, Creatore)

Lavoratore(CF, Nome, Cognome, Ruolo)

Messaggio(CF, DataInvio, OrarioInvio, Contenuto, IDCanale, IDProgetto)

Risposta(Rispondente, DataInvioRispondente, OrarioInvioRispondente, Mittente, DataInvioMittente, OrarioInvioMittente, Visibilità)

Partecipazione(CF, IDProgetto)

Appartenenza(CF, IDCanale, IDProgetto)

Vincoli:

Progetto(CapoProgetto) $\subseteq$ Lavoratore(CF)

CanaleDiComunicazione(Creatore) $\subseteq$ Lavoratore(CF)

CanaleDiComunicazione(IDProgetto) $\subseteq$ Progetto(ID)

Messaggio(CF) $\subseteq$ Lavoratore(CF)

Messaggio(IDCanale, IDProgetto) $\subseteq$ CanaleDiComunicazione(IDCanale, IDProgetto)

Risposta(Mittente, DataInvioMittente, OrarioInvioMittente) $\subseteq$ Messaggio(CF, DataInvio, OrarioInvio)

Risposta(Rispondente, DataInvioRispondente, OrarioInvioRispondente) $\subseteq$ Messaggio(CF, DataInvio, OrarioInvio)

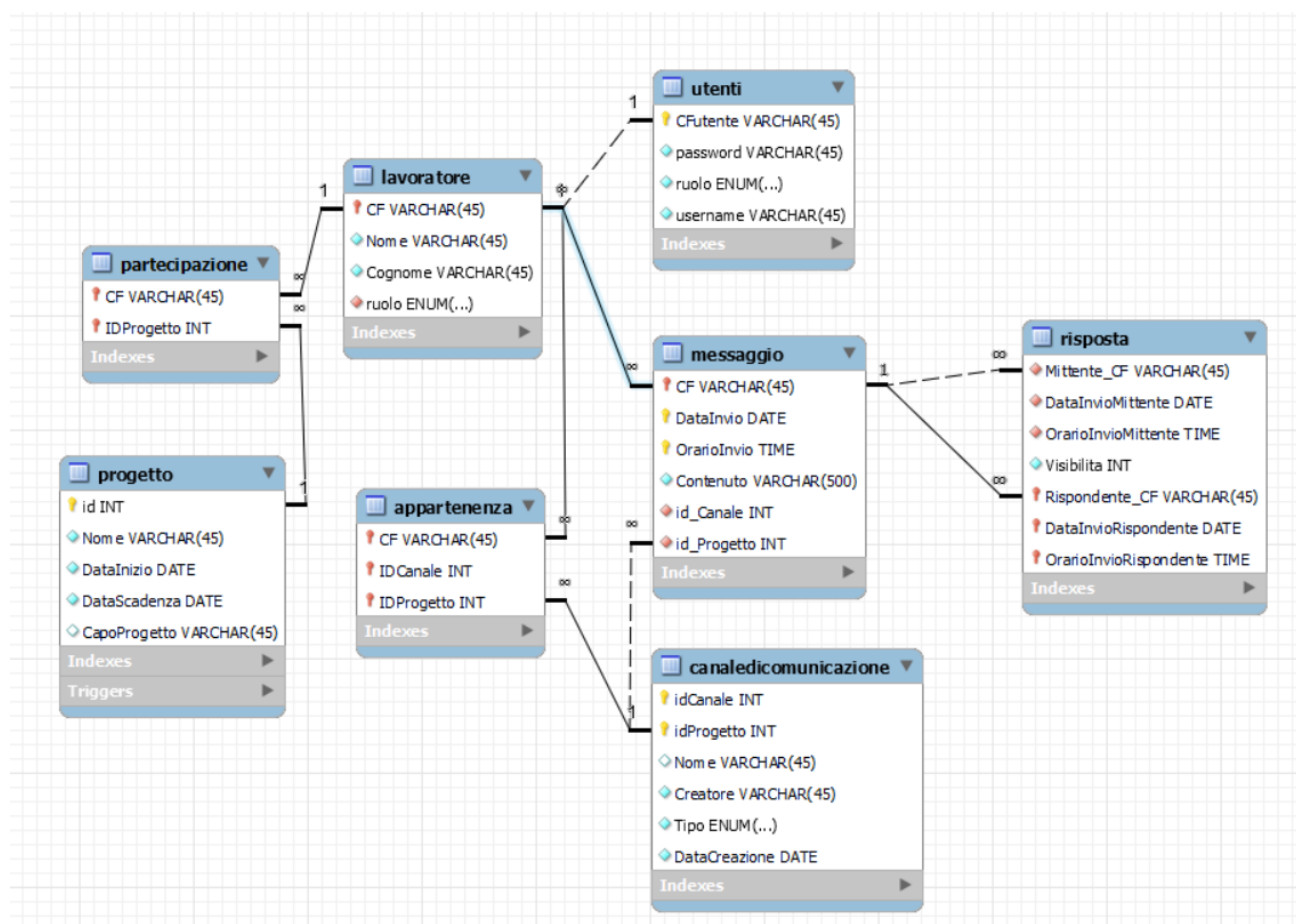
Partecipazione(CF) $\subseteq$ Lavoratore(CF)

Partecipazione(IDProgetto) $\subseteq$ Progetto(ID)

Appartenenza(CF) $\subseteq$ Lavoratore(CF)

Appartenenza(IDCanale, IDProgetto) $\subseteq$ CanaleDiComunicazione(IDCanale, IDProgetto)

La rappresentazione grafica del modello:



Normalizzazione del modello relazionale

Il modello relazionale è in forma normale di Boyce e Codd. Lo andiamo a dimostrare, verificando che tutte le relazioni soddisfano la forma normale di Boyce e Codd.

Iniziamo con le relazioni che hanno come attributi solamente quelli che formano la chiave primaria, tra cui PARTECIPAZIONE e APPARTENENZA, che sono, ovviamente, in forma normale.

Adesso analizziamo le altre:

PROGETTO: Questa relazione presenta come dipendenza funzionale: $ID \rightarrow \text{Nome, DataInizio, DataScadenza, CapoProgetto}$ ed è in forma normale, in quanto gli attributi sono determinati solamente dalla chiave primaria.

CANALEDICOMUNICAZIONE: Questa relazione presenta come dipendenza funzionale: $IDCanale, IDProgetto \rightarrow \text{Nome, Creatore}$ ed è in forma normale, in quanto tutti gli attributi (Nome, Creatore) dipendono solamente dalla chiave composta.

LAVORATORE: Questa relazione presenta come dipendenza funzionale: $CF \rightarrow \text{Nome, Cognome, Ruolo}$ ed è in forma normale.

MESSAGGIO: Questa relazione presenta come dipendenza funzionale: CF, DataInvio, OrarioInvio → Contenuto, IDCanale, IDProgetto ed è in forma normale.

RISPOSTA: Questa relazione presenta come dipendenza funzionale: Rispondente, DataInvioRispondente, OrarioInvioRispondente → Mittente, DataInvioMittente, OrarioInvioMittente, Visibilità ed è in forma normale.

5. Progettazione fisica

Utenti e privilegi

Per permettere l'identificazione degli utenti è stata introdotta una tabella "utenti" per memorizzarne le credenziali. Nell'applicazione sono previste quattro tipi di utenti:

- Ruolo Login: si occupa dell'accesso al sistema.
- Ruolo Amministratore: questo utente si occupa della coordinazione dei progetti e dei capi progetti.
- Ruolo Capo progetto: questo utente si occupa della creazione e della coordinazione dei canali di comunicazione di un progetto, inoltre ha il privilegio di mandare i messaggi all'interno di canali di comunicazione a cui fa parte.
- Ruolo Dipendente: questo utente ha il privilegio di mandare i messaggi all'interno di canali di comunicazione a cui fa parte.

I diversi utenti hanno grant in esecuzione sulle seguenti procedure:

Utente	Procedure
login	login
amministratore_chat	assegna_capo inserisci_lavoratore inserisci_progetto lista_progetti_senza_capo lista_capi lista_progetti_con_capo
capoprogetto_chat	assegnazione_canale assegnazione_progetto conversazione_by_idCanale_idProgetto inserisci_nuovo_canale inserisci_messaggio inserisci_risposta inserisci_risposta_privata lista_canali_public_by_IDprogetto lista_canali_by_IDprogetto lista_canali_privati_by_IDprogetto lista_progetti_coordinato recupero_lavoratori_by_IdProgetto recupero_lavoratori_by_IdCanale recupero_messaggio_originario recupero_tutti_lavoratori recupero_tutti_canali_appartenenti
dipendente_chat	conversazione_by_idCanale_idProgetto inserisci_messaggio

	inserisci_risposta inserisci_risposta_privata lista_canali_by_IDprogetto lista_progetti_appartenenti recupero_tutti_canali_appartenenti recupero_lavoratori_by_IdCanale recupero_messaggio_originario
--	---

Strutture di memorizzazione

Tabella <Utenti>		
Colonna	Tipo di dato	Attributi ²
CFutente	Varchar(45)	PK, NN
Password	Varchar(45)	NN
Ruolo	ENUM('Amministratore', 'Capoprogetto', 'Dipendente')	NN
Username	Varchar(45)	NN

Tabella <Lavoratore>		
Colonna	Tipo di dato	Attributi
CF	Varchar(45)	PK, NN
Nome	Varchar(45)	NN
Cognome	Varchar(45)	NN
Ruolo	ENUM('Amministratore', 'Capoprogetto', 'Dipendente')	NN

Tabella <Progetto>		
Colonna	Tipo di dato	Attributi
ID	INT	PK, NN, UQ
Nome	Varchar(45)	NN
DataInizio	Date	NN
DataScadenza	Date	NN
CapoProgetto	Varchar(45)	

Tabella <Canale di comunicazione>		
Colonna	Tipo di dato	Attributi
IDCanale	INT	PK, NN, AI
IDProgetto	INT	PK, NN
Nome	Varchar(45)	
Creatore	Varchar(45)	NN
Tipo	ENUM('Public', 'Private')	NN
DataCreazione	Date	NN

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella <Messaggio>		
Colonna	Tipo di dato	Attributi
CF	Varchar(45)	PK, NN
DataInvio	Date	PK, NN
OrarioInvio	Time	PK, NN
Contenuto	Varchar(500)	NN
ID_Canale	INT	NN
ID_Progetto	INT	NN

Tabella <Risposta >		
Colonna	Tipo di dato	Attributi
Mittente_CF	Varchar(45)	NN
DataInvioMittente	Date	NN
OrarioInvioMittente	Time	NN
Visibilità	INT	NN
Rispondente_CF	Varchar(45)	PK, NN
DataInvioRispondente	Date	PK, NN
OrarioInvioRispondente	Time	PK, NN

Tabella <Appartenenza >		
Colonna	Tipo di dato	Attributi
CF	Varchar(45)	PK, NN
IDCanale	INT	PK, NN
IDProgetto	INT	PK, NN

Tabella <Partecipazione >		
Colonna	Tipo di dato	Attributi
CF	Varchar(45)	PK, NN
IDProgetto	INT	PK, NN

Indici

Si omettono gli indici di tipo PRIMARY per le chiavi e INDEX per le foreign key, in quanto generati automaticamente.

Trigger

È stato introdotto un trigger BEFORE INSERT per realizzare una regola aziendale.

Tabella progetto: BEFORE INSERT

L'attributo DataScadenza della tabella "progetto" indica la data di scadenza di un progetto, per cui nel momento di inserimento di un nuovo progetto, la DataScadenza deve essere posteriore rispetto alla DataInizio. Il seguente trigger, ad ogni inserimento, controlla se le date hanno rispettato la regola aziendale.

```
CREATE DEFINER=`root`@`localhost` TRIGGER `progetto_BEFORE_INSERT` BEFORE
INSERT ON `progetto` FOR EACH ROW BEGIN
    -- Controlla che DataScadenza sia posteriore a DataInizio
    IF NEW.DataScadenza <= NEW.DataInizio THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La DataScadenza deve essere posteriore alla DataInizio';
    END IF;
END
```

Eventi

È stato istanziato un evento per la rimozione dei dati relativi ai progetti terminati da più di 3 anni. L'evento viene schedulato per la prima volta per il 1° settembre del anno 2024, poi il controllo avverrà al primo giorno di ogni mese successivo a tale giorno di inizio.

```
SET GLOBAL event_scheduler = on;
```

```
CREATE EVENT elimina_progetti_obsoleti
ON SCHEDULE
    EVERY 1 MONTH
STARTS '2024-09-01 00:00:00' -- Specifica l'inizio dell'evento
ON COMPLETION PRESERVE
DO BEGIN
    DELETE FROM progetto
    WHERE DataScadenza <= DATE_SUB(CURDATE(), INTERVAL 3 YEAR);
```

Viste

Non sono state utilizzate viste.

Stored Procedures e transazioni

assegna_capo

Procedura per assegnare il capo progetto a un progetto senza capo progetto.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `assegna_capo`(in id_progetto int, in  
cf_capo varchar(45))
```

```
BEGIN
```

```
    declare exit handler for sqlexception
```

```
    begin
```

```
        rollback;
```

```
        resignal;
```

```
end;
```

```
    set transaction isolation level repeatable read;
```

```
    start transaction;
```

```
    -- Aggiorna la colonna CapoProgetto nella tabella progetto
```

```
    UPDATE progetto
```

```
    SET CapoProgetto = cf_capo
```

```
    WHERE id = id_progetto;
```

```
    commit;
```

```
END
```

assegnazione_canale

Procedura per assegnare il canale a un lavoratore.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `assegnazione_canale`(in var_cf  
varchar(45), in id_canale int, in id_progetto int)
```

```
BEGIN
```

```
    declare exit handler for sqlexception
```

```
begin
    rollback;
    resignal;
end;

set transaction isolation level read uncommitted;
start transaction;

insert into appartenenza(CF, IDCanale, IDProgetto)
values (var_cf, id_canale, id_progetto);

commit;
END
```

assegnazione_progetto

Procedura per assegnare un progetto a un lavoratore.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `assegnazione_progetto`(in cf_lavoratore
varchar(45), in id_progetto int)
```

```
BEGIN
    declare exit handler for sqlexception
begin
    rollback;
    resignal;
end;

set transaction isolation level read uncommitted;
start transaction;

insert into partecipazione
values (cf_lavoratore, id_progetto);

commit;
END
```

conversazione_by_idCanale_idProgetto

Procedura per recuperare i messaggi di un canale appartenente a un progetto.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `conversazione_by_idCanale_idProgetto`(in  
idcanale int, in idprogetto int)
```

```
BEGIN
```

```
    declare exit handler for sqlexception
```

```
    begin
```

```
        rollback;
```

```
        resignal;
```

```
    end;
```

```
    set transaction isolation level repeatable read;
```

```
    start transaction;
```

```
    select m.CF, l.Nome, l.Cognome, m.DataInvio, m.OrarioInvio, m.Contenuto, r.Visibilita,  
    r.Rispondente_CF, r.Mittente_CF, r.DataInvioMittente, r.OrarioInvioMittente
```

```
    from messaggio m
```

```
    JOIN lavoratore l on m.CF = l.CF
```

```
    LEFT JOIN
```

```
        risposta r on m.CF = r.Rispondente_CF
```

```
            and m.DataInvio = r.DataInvioRispondente
```

```
            and m.OrarioInvio = r.OrarioInvioRispondente
```

```
    where idcanale = m.id_Canale and idprogetto = m.id_Progetto
```

```
    order by m.DataInvio, m.OrarioInvio;
```

```
    if FOUND_ROWS() = 0
```

```
        then
```

```
            signal sqlstate '45000' set message_text = "Non ci sono messaggi disponibili";
```

```
    end if;
```

```
    commit;
```

```
END
```


inserisci_lavoratore

Procedura per inserire un nuovo lavoratore nel sistema.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `inserisci_lavoratore`(  
in var_cf varchar(45),  
in var_nome varchar(45),  
in var_cognome varchar(45),  
in var_ruolo int,  
in var_username varchar(45),  
in var_password varchar(45))  
BEGIN
```

```
    declare exit handler for sqlexception  
    begin  
        rollback; -- rollback any changes made in the transaction  
        resignal; -- raise again the sql exception to the caller  
    end;
```

```
    set transaction isolation level read uncommitted;  
    start transaction;
```

```
-- inserimento utente
```

```
insert into `utenti` (`CFutente`, `password`, `ruolo`, `username`)  
values (var_cf, MD5(var_password), var_ruolo, var_username);
```

```
-- inserimento lavoratore
```

```
insert into `lavoratore` (`CF`, `Nome`, `Cognome`, `ruolo`)  
values (var_cf, var_nome, var_cognome, var_ruolo);
```

```
commit;
```

```
END
```

inserisci_messaggio

Procedura per inserire un nuovo messaggio in un canale di un progetto.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `inserisci_messaggio`(in cf_utente
varchar(45), in data_invio date, in orario_invio time, in meg varchar(500), in id_canale int, in
id_progetto int)
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;

    set transaction isolation level read uncommitted;
    start transaction;

    insert into messaggio(CF, DataInvio, OrarioInvio, Contenuto, id_Canale, id_Progetto)
    values (cf_utente, data_invio, orario_invio, meg, id_canale, id_progetto);

    commit;
END
```

inserisci_nuovo_canale

Procedura per inserire un nuovo canale in un progetto.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `inserisci_nuovo_canale`(in id_progetto int,
in nome_canale varchar(45), in cf varchar(45), in tipo_canale int, in data_creazione Date)
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;

    set transaction isolation level read uncommitted;
    start transaction;
```

```
-- inserimento canale pubblico
insert into canaledicomunicazione(idProgetto, Nome, Creatore, Tipo, DataCreazione)
values (id_progetto, nome_canale, cf, tipo_canale, data_creazione);

-- Recupero del valore auto-incrementato di IDCanale
set @idCanale = LAST_INSERT_ID();

-- Inserimento nella tabella appartenenza usando il valore auto-incrementato di IDCanale
insert into appartenenza(CF, IDCanale, IDProgetto)
values (cf, @idCanale, id_progetto);

commit;
END
```

inserisci_progetto

Procedura per inserire un nuovo progetto.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `inserisci_progetto`(in p_id int, in p_nome
varchar(45), in p_data_inizio date, in p_data_scadenza date, p_cf_capo varchar(45))
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;

    set transaction isolation level read uncommitted;
    start transaction;

    INSERT INTO progetto (id, Nome, DataInizio, DataScadenza, CapoProgetto)
    VALUES (p_id, p_nome, p_data_inizio, p_data_scadenza, p_cf_capo);

    commit;
END
```

inserisci_risposta

Procedura per inserire una risposta riferita a un messaggio di un canale.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `inserisci_risposta`(in cf_m varchar(45), in
data_m date, in ora_m time, in vis int, in cf_r varchar(45), in data_r date, in ora_r time)
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;

    set transaction isolation level read uncommitted;
    start transaction;

    insert into risposta(Mittente_CF, DataInvioMittente, OrarioInvioMittente, Visibilita,
Rispondente_CF, DataInvioRispondente, OrarioInvioRispondente)
    values (cf_m, data_m, ora_m, vis, cf_r, data_r, ora_r);

    commit;
END
```

inserisci_risposta_privata

Procedura per inserire il messaggio di risposta privata a un messaggio originario che si trova sul canale pubblico.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `inserisci_risposta_privata`(
in cf_rispondente varchar(45),
in data_invio_risp date,
in orario_invio_risp time,
in risposta_privata varchar(500),
in id_canale_originario int,
in id_progetto_originario int,
in cf_mittente_originario varchar(45),
in data_invio_originario date,
```

in orario_invio_originario time)

BEGIN

declare exit handler for sqlexception

begin

rollback;

resignal;

end;

set transaction isolation level read uncommitted;

start transaction;

-- Creazione canale privata per la risposta privata

INSERT INTO canaledicomunicazione (idProgetto, Creatore, Tipo, DataCreazione)

VALUES (id_progetto_originario, cf_rispondente, 'Private', data_invio_risp);

-- Recupero del valore auto-incrementato di IDCanale

SET @idCanale = LAST_INSERT_ID();

-- Assegna il canale ai due partecipanti

INSERT INTO appartenenza (`CF`, `IDCanale`, `IDProgetto`)

VALUES (cf_rispondente, @idCanale, id_progetto_originario);

INSERT INTO appartenenza (`CF`, `IDCanale`, `IDProgetto`)

VALUES (cf_mittente_originario, @idCanale, id_progetto_originario);

-- Inserimento del messaggio di risposta nella tabella 'messaggio'

INSERT INTO `messaggio` (`CF`, `DataInvio`, `OrarioInvio`, `Contenuto`, `id\_Canale`,
`id\_Progetto`)

VALUES (cf_rispondente, data_invio_risp, orario_invio_risp, risposta_privata, @idCanale,
id_progetto_originario);

-- Inserimento della risposta privata nella tabella 'risposta'

INSERT INTO `risposta` (`Mittente\_CF`, `DataInvioMittente`, `OrarioInvioMittente`, `Visibilita`,
`Rispondente\_CF`, `DataInvioRispondente`, `OrarioInvioRispondente`)

```
VALUES (cf_mittente_originario, data_invio_originario, orario_invio_originario, 1,  
cf_rispondente, data_invio_risp, orario_invio_risp);
```

```
commit;
```

```
END
```

lista_canali_by_IDprogetto

Procedura per recuperare tutti i canali appartenenti al IDprogetto fornito.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `lista_canali_by_IDprogetto`(in cf_utente  
varchar(45), in id_progetto int)
```

```
BEGIN
```

```
    declare exit handler for sqlexception
```

```
begin
```

```
    rollback;
```

```
    resignal;
```

```
end;
```

```
set transaction isolation level read committed;
```

```
start transaction;
```

```
select appartenenza.IDCanale, C.Nome, C.Creatore, C.Tipo, C.DataCreazione
```

```
from appartenenza join canaledicomunicazione as C on C.idCanale = appartenenza.IDCanale and  
C.idProgetto = appartenenza.IDProgetto
```

```
where appartenenza.IDProgetto = id_progetto and cf = cf_utente;
```

```
if FOUND_ROWS() = 0
```

```
then
```

```
    signal sqlstate '45000' set message_text = "Non ci sono canali a cui fa parte";
```

```
end if;
```

```
commit;
```

```
END
```

lista_canali_privati_by_IDprogetto

Procedura per recuperare una lista di canali privati appartenenti al IDProgetto fornito.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `lista_canali_privati_by_IDprogetto`(in  
cf_capo varchar(45), in id_progetto int)
```

```
BEGIN
```

```
    declare exit handler for sqlexception
```

```
begin
```

```
    rollback;
```

```
    resignal;
```

```
end;
```

```
set transaction isolation level read committed;
```

```
start transaction;
```

```
-- Seleziona i canali privati dove cf_capo non è né il creatore né un membro
```

```
SELECT idCanale, idProgetto, Creatore, Tipo, DataCreazione
```

```
FROM canaledicomunicazione
```

```
WHERE Tipo = 'Private'
```

```
AND Creatore != cf_capo
```

```
AND idProgetto = id_progetto
```

```
AND idCanale NOT IN (
```

```
    SELECT IDCanale
```

```
    FROM appartenenza
```

```
    WHERE CF = cf_capo
```

```
);
```

```
if FOUND_ROWS() = 0
```

```
    then
```

```
        signal sqlstate '45000' set message_text = "Non ci sono canali privati disponibili";
```

```
end if;
```

```
commit;
```

```
END
```

lista_canali_public_by_IDprogetto

Procedura per recuperare una lista di canali pubblici appartenenti al IDprogetto fornito.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `lista_canali_public_by_IDprogetto`(in
cf_utente varchar(45), in id_progetto int)
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;

    set transaction isolation level read committed;
    start transaction;

    select appartenenza.IDCanale, C.Nome, C.Creatore, C.Tipo, C.DataCreazione
    from appartenenza join canaledicomunicazione as C on C.idCanale = appartenenza.IDCanale and
C.idProgetto = appartenenza.IDProgetto
    where appartenenza.IDProgetto = id_progetto and cf = cf_utente and Tipo = 'Public';

    if FOUND_ROWS() = 0
    then
        signal sqlstate '45000' set message_text = "Non ci sono canali a cui fa parte";
    end if;

    commit;
END
```

lista_capi

Procedura per recuperare una lista di capi progetto.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `lista_capi`()
BEGIN
    declare exit handler for sqlexception
    begin
```



```
        rollback;
        resignal;
    end;

    set transaction isolation level read committed;
    start transaction;

    select CF, Nome, Cognome
    from lavoratore
    where ruolo = 2;

    if FOUND_ROWS() = 0
    then
        signal sqlstate '45000' set message_text = "Non ci sono capi progetto candidati";
    end if;

    commit;
END
```

lista_progetti_appartenenti

Procedura per recuperare una lista di progetti a cui appartiene il lavoratore loggato.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `lista_progetti_appartenenti`(in var_cf
varchar(45))
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;

    set transaction isolation level read committed;
    start transaction;
```

```
select IDProgetto, Nome, DataInizio, DataScadenza, CapoProgetto
from partecipazione join progetto on id = IDProgetto
where CF = var_cf;
```

```
if FOUND_ROWS() = 0
    then
        signal sqlstate '45000' set message_text = "Non ci sono progetti a cui appartieni";
    end if;
```

```
commit;
END
```

lista_progetti_con_capo

Procedura per recuperare una lista di progetti con capi progetto assegnati.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `lista_progetti_con_capo`()
```

```
BEGIN
```

```
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;
```

```
set transaction isolation level read committed;
start transaction;
```

```
select id, Nome, DataInizio, DataScadenza, CapoProgetto
from progetto
where CapoProgetto is not null;
```

```
if FOUND_ROWS() = 0
    then
        signal sqlstate '45000' set message_text = "Non ci sono progetti con capo progetto";
    end if;
```

```
commit;  
END
```

lista_progetti_coordinato

Procedura per recuperare una lista di progetti coordinati dal lavoratore loggato con ruolo Capo progetto.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `lista_progetti_coordinato`(in var_cf  
varchar(45))
```

```
BEGIN
```

```
    declare exit handler for sqlexception
```

```
    begin
```

```
        rollback;
```

```
        resignal;
```

```
    end;
```

```
    set transaction isolation level read committed;
```

```
    start transaction;
```

```
    select id, Nome, DataInizio, DataScadenza
```

```
    from progetto
```

```
    where CapoProgetto = var_cf;
```

```
    if FOUND_ROWS() = 0
```

```
        then
```

```
            signal sqlstate '45000' set message_text = "Non ci sono progetti coordinati da te";
```

```
    end if;
```

```
    commit;
```

```
END
```

lista_progetti_senza_capo

Procedura per recuperare una lista di progetti senza capo progetto assegnato.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `lista_progetti_senza_capo`()
BEGIN
declare exit handler for sqlexception
begin
    rollback;
    resignal;
end;

set transaction isolation level read committed;
start transaction;

select id, Nome, DataInizio, DataScadenza, CapoProgetto
from progetto
where CapoProgetto is null;

if FOUND_ROWS() = 0
then
    signal sqlstate '45000' set message_text = "Non ci sono progetti senza capo progetto";
end if;

commit;
END
```

login

Procedura per effettuare login, la quale riceve in input uno CF e una password e seleziona il ruolo dell'utente corrispondente, se esiste, nella tabella utenti. A seconda del ruolo, viene settata la variabile di output ad un determinato valore intero, attraverso il quale sarà selezionato il ruolo corrispondente all'interno del client.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `login`(in var_CF varchar(45), in
var_password varchar(45), out var_role INT, out var_username varchar(45))
BEGIN
```

```
declare var_ruolo_utente ENUM('Amministratore', 'Capoprogetto', 'Dipendente');
```

```
-- Ottieni il ruolo dell'utente  
select `ruolo`  
from `utenti`  
where `CFutente` = var_CF  
and `password` = md5(var_password)  
into var_ruolo_utente;
```

```
-- Ottieni il username dell'utente  
select `username`  
from `utenti`  
where `CFutente` = var_CF  
and `password` = md5(var_password)  
into var_username;
```

```
-- Controlla il ruolo e imposta il valore di var_role  
if var_ruolo_utente = 'Amministratore' then  
    set var_role = 1;  
elseif var_ruolo_utente = 'Capoprogetto' then  
    set var_role = 2;  
elseif var_ruolo_utente = 'Dipendente' then  
    set var_role = 3;  
else  
    set var_role = 4;  
end if;
```

```
END
```

recupero_lavoratori_by_IdCanale

Procedura per recuperare una lista di lavoratori che appartengono al IDCanale fornito.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `recupero_lavoratori_by_IdCanale`(in
id_canale int ,in id_progetto int)
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;

    set transaction isolation level read committed;
    start transaction;

    select L.CF, L.Nome, L.Cognome, L.ruolo
    from appartenenza A join lavoratore L on A.CF = L.CF
    where A.IDProgetto = id_progetto and A.IDCanale = id_canale;

    if FOUND_ROWS() = 0
    then
        signal sqlstate '45000' set message_text = "Non ci sono lavoratori appartenenti al canale
fornito";
    end if;

    commit;
END
```

recupero_lavoratori_by_IdProgetto

Procedura per recuperare una lista di lavoratori appartenenti al IDProgetto fornito.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `recupero_lavoratori_by_IdProgetto`(in
id_progetto int)
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
```

```
        resignal;
    end;

    set transaction isolation level read committed;
    start transaction;

    select L.CF, L.Nome, L.Cognome, L.ruolo
    from partecipazione P join lavoratore L on P.CF = L.CF
    where P.IDProgetto = id_progetto;

    if FOUND_ROWS() = 0
    then
        signal sqlstate '45000' set message_text = "Non ci sono lavoratori appartenenti al progetto
fornito";
    end if;

    commit;
END
```

recupero_messaggio_originario

Procedura per recuperare il messaggio originario a cui il messaggio di risposta è riferito.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `recupero_messaggio_originario`(
in cf_rispondente varchar(45),
in dataInvioRispondente date,
in orarioInvioRispondente time
)
BEGIN
    declare exit handler for sqlexception
    begin
        rollback;
        resignal;
    end;
```

```
set transaction isolation level repeatable read;
start transaction;

select distinct Mittente_CF, DataInvioMittente, OrarioInvioMittente, Contenuto, Nome, Cognome
from messaggio
    join risposta
        on CF = Mittente_CF and DataInvio = DataInvioMittente and OrarioInvio =
OrarioInvioMittente
    join lavoratore on messaggio.CF = lavoratore.CF
where Rispondente_CF = cf_rispondente and DataInvioRispondente = dataInvioRispondente and
OrarioInvioRispondente = orarioInvioRispondente and Visibilita = 1
order by DataInvioMittente, OrarioInvioRispondente;

commit;
END
```

recupero_tutti_canali_appartenenti

Procedura per recuperare tutti i canali a cui appartiene il lavoratore loggato.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `recupero_tutti_canali_appartenenti`(in
cf_utente varchar(45))
```

```
BEGIN
```

```
    declare exit handler for sqlexception
```

```
begin
```

```
    rollback;
```

```
    resignal;
```

```
end;
```

```
set transaction isolation level read committed;
```

```
start transaction;
```

```
select appartenenza.IDCanale, appartenenza.IDProgetto, C.Nome, C.Creatore, C.Tipo,
C.DataCreazione
```



```
from appartenenza join canaledicomunicazione as C on C.idCanale = appartenenza.IDCanale and
C.idProgetto = appartenenza.IDProgetto
where cf = cf_utente;

if FOUND_ROWS() = 0
then
    signal sqlstate '45000' set message_text = "Non ci sono canali a cui fa parte";
end if;

commit;
END
```

recupero_tutti_lavoratori

Procedura per recuperare tutti i lavoratori di ruolo Dipendente.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `recupero_tutti_lavoratori`()
BEGIN
    declare exit handler for sqlexception
begin
    rollback;
    resignal;
end;

set transaction isolation level read committed;
start transaction;

select CF, Nome, Cognome, ruolo
from lavoratore
where ruolo = 'Dipendente';

if FOUND_ROWS() = 0
then
    signal sqlstate '45000' set message_text = "Non ci sono lavoratori";
end if;
```

```
    commit;  
END
```